

课程笔记

机器学习基本概念简介

李宏毅《机器学习》2025 · 第一节（上）

基于李宏毅老师课程整理（lecture-to-notes 自动生成）

2026 年 6 月 4 日



视频作者/频道：李宏毅深度学习课堂（Bilibili 搬运）

发布日期：2025-08-06

视频时长：约 50 分钟（49:59）

视频链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=1>

目录

1	引言：机器学习在寻找什么	2
1.1	本章小结	3
2	函数的三种类型	3
2.1	本章小结	4
3	机器学习的三个步骤	4
3.1	第一步：带有未知参数的函数 (Model)	4
3.2	第二步：从训练资料定义损失 (Loss)	5
3.3	第三步：最佳化 (Optimization)	7
3.4	本章小结	9
4	从训练到预测：泛化的考验	10
5	让模型更有弹性：线性模型	11
5.1	本章小结	13
6	总结与延伸	13

1 引言：机器学习在寻找什么

机器学习（Machine Learning）常被科普文章渲染得“玄之又玄”——仿佛机器一旦学会学习，就有了人工智能，接下来就要统治人类。但若用一句话概括它的本质，其实相当朴素。

机器学习的本质

机器学习就是让机器具备“**找一个函数**”的能力。这个函数往往极其复杂，复杂到人类无法用手写的方程式直接表达，于是我们寄希望于借助机器的力量，从资料中把它**自动**找出来。

以语音辨识为例：我们想要的是一个函数 f ，输入是一段声音讯号，输出是这段声音对应的文字内容。这样的函数无比复杂，绝不是人能手写出来的方程式。



图 1: 机器学习的核心是寻找一个人类难以手写的复杂函数。图中以语音辨识为例：函数输入声音波形，输出文字 “How are you”¹

这样的任务比比皆是，它们的共同点都是“找一个复杂的函数”：

- **影像辨识**： $f(\text{一张图片}) = \text{图片里的内容}$ ；
- **下围棋 (AlphaGo)**： $f(\text{棋盘上黑子与白子的位置}) = \text{下一步应落子的位置}$ 。

为什么要“学”出函数，而不是直接写

像“声音讯号 → 文字”“棋盘 → 落子”这样的函数，输入输出关系极度复杂，人类没有能力把它显式地写成方程式。机器学习的价值，正是在人写不出函数时，凭借资料与算力把它逼近出来。

¹视频画面时间区间：00:02:00–00:02:15。

1.1 本章小结

机器学习 $\approx$ 寻找函数。当任务的输入输出关系复杂到无法手写时，就交给机器从资料中自动寻找这个函数——这是贯穿全课程的第一性原理。

2 函数的三种类型

随着要找的函数不同，机器学习的任务可以分成几大类。这里先认识三个常见的专有名词。

- **回归 (Regression)**：函数的输出是一个**数值** (scalar)。例如预测明天中午的 PM2.5——输入今天的 PM2.5、平均温度、臭氧浓度等指标，输出一个数值。
- **分类 (Classification)**：人类预先准备好若干**选项** (又称类别 class)，函数从中选一个作为输出。例如 Gmail 侦测一封邮件“是 / 不是”垃圾邮件 (两个选项)；AlphaGo 本质上也是分类——棋盘有 $19 \times 19 = 361$ 个位置，相当于一道有 361 个选项的选择题。
- **结构化学习 (Structured Learning)**：机器要产生一个**有结构**的物件，例如画一张图、写一篇文章。用拟人化的说法，就是让机器学会“创造”。

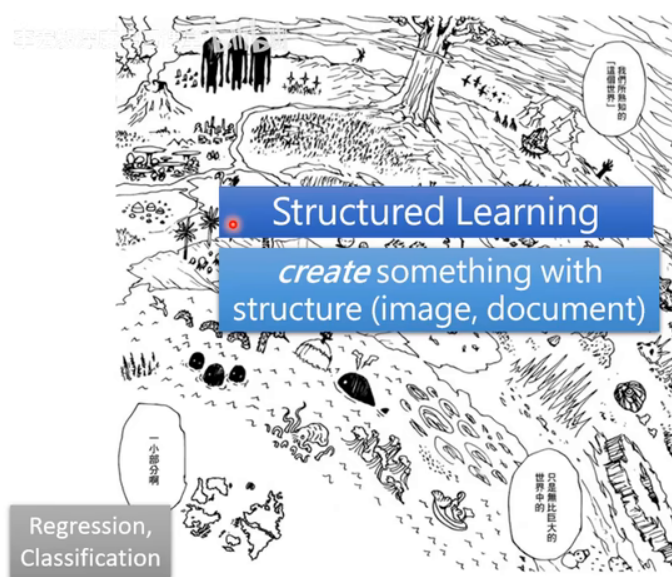


图 2: 结构化学习 (Structured Learning): 让机器创造出有结构的产物 (图片、文档)。它位于回归与分类 (图左下角) 之外，是更具挑战性的一类任务²

回归与分类之外的“黑暗大陆”

很多教科书讲到 Regression 和 Classification 就停下了，仿佛机器学习的世界只有“两大洲”。但在它们之外还有一块常被忽视的领域——Structured Learning, 让机器创造有结构的输出。(李宏毅老师在此用《全职猎人》中的“黑暗大陆”作了一个比喻。)

² 视频画面时间区间：00:08:15–00:08:30。

2.1 本章小结

按输出形态，机器学习任务分为三类：回归（输出数值）、分类（从选项中选择）、结构化学习（创造有结构的物件）。同一个问题（如下围棋）既可看作分类，也启发我们：任务类型决定了我们要找的是什么样的函数。

3 机器学习的三个步骤

确立了“机器学习 = 找函数”之后，真正的问题是：机器怎么找出这个函数？李宏毅老师以“预测自己的 YouTube 频道隔天的总观看人数”为贯穿案例，把找函数的过程拆解成三个步骤。

3.1 第一步：带有未知参数的函数 (Model)

第一步，我们先**猜测**函数的数学形式——这个猜测来自我们对问题的理解，即领域知识 (Domain Knowledge)。对于观看人数预测，一个最初步的猜测是：

$$y = b + w x_1$$

其中各符号的含义为：

- y ：要预测的目标，例如 2 月 26 日当天的总观看人数（今天尚未过完，所以未知）；
- x_1 ：已知的输入，即**特征** (feature) ——前一天（2 月 25 日）的观看人数；
- w ：与特征相乘的未知参数，称为**权重** (weight)；
- b ：直接相加的未知参数，称为**偏置** (bias)。

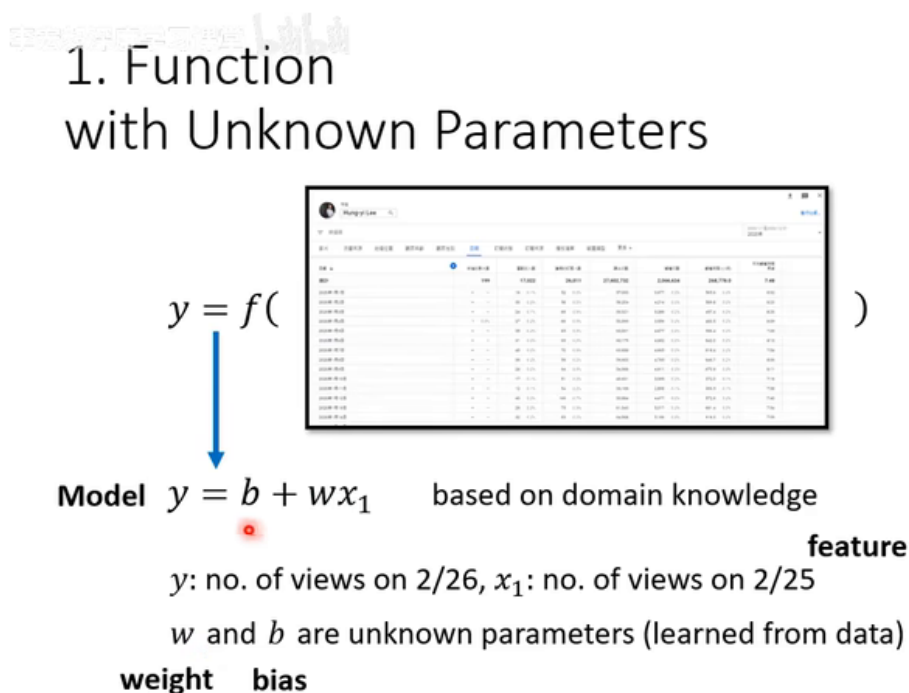


图 3: 第一步——写出带有未知参数的函数 $y = b + w x_1$ 。其中 x_1 是特征（前一天观看数）， w 、 b 是待从资料中学出的未知参数（权重与偏置）³

这种带有未知参数 (unknown parameters) 的函数，在机器学习中就叫做**模型** (model)。这里之所以猜测 $y = b + wx_1$ ，背后的直觉是：今天的观看数大概会和昨天的有关联，但不会一模一样，于是把昨天的数值乘上 w 再加一个修正项 b 。

Domain Knowledge 用在哪里

经常有人说“做机器学习需要一些领域知识”。它最主要的用武之地，正是在第一步**写出带有未知参数的函数**时——你对问题了解越深，越能猜出一个合理的函数形式 (model)。

3.2 第二步：从训练资料定义损失 (Loss)

第二步，定义一个叫做**损失** (Loss) 的函数 $L(b, w)$ 。注意它本身也是一个函数，但它的**输入是模型的参数** b 、 w ，输出是一个数值，用来衡量“这一组参数好不好”。

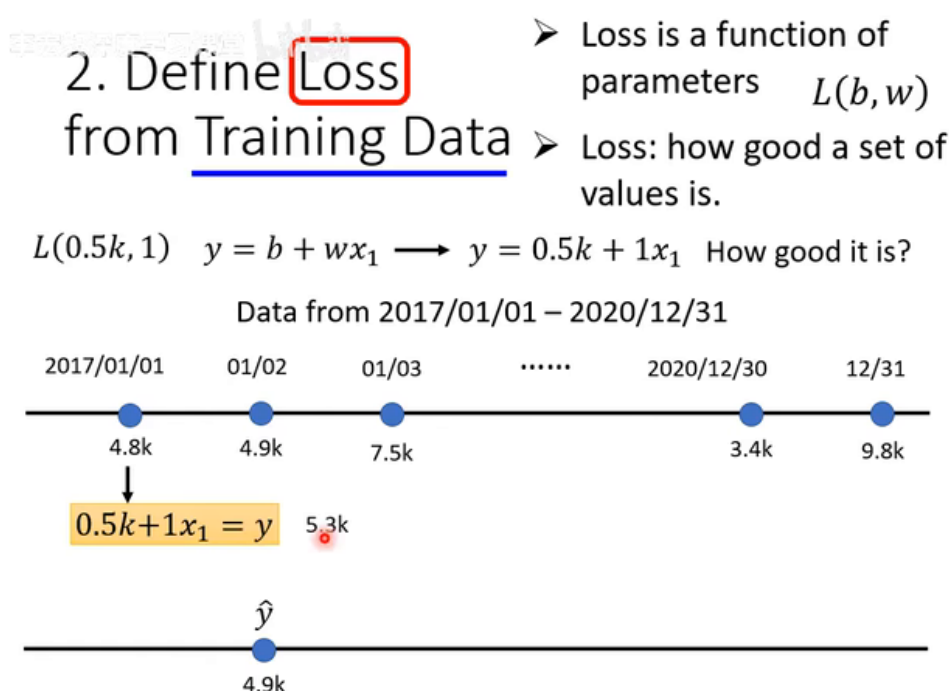


图 4: 损失的计算示例：固定 $b = 0.5k$ 、 $w = 1$ 时，用 1 月 1 日的真实值 4.8k 预测 1 月 2 日得 5.3k，而真实值（标签） $\hat{y} = 4.9k$ ⁴

损失要用**训练资料**来计算。本例的训练资料是该频道 2017/01/01 至 2020/12/31 共三年每天的真实观看数。具体做法：先固定一组参数（如 $b = 0.5k$ 、 $w = 1$ ），模型变为 $y = 0.5k + x_1$ ；把 1 月 1 日的真实值 4.8k 代入，预测 1 月 2 日为 5.3k，而 1 月 2 日的真实值（标签 label）是 4.9k，二者的差距为

$$e_1 = |y - \hat{y}| = |4.9k - 5.3k| = 0.4k.$$

对三年里的每一天都这样算出误差 e_n ，再取平均，就得到损失：

$$L = \frac{1}{N} \sum_n e_n$$

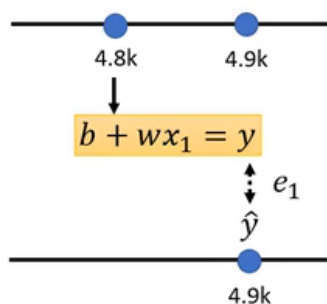
³视频画面时间区间：00:14:45–00:15:00。

⁴视频画面时间区间：00:18:30–00:18:45。

- N : 训练资料的笔数 (三年约 365×3 笔);
- e_n : 第 n 笔资料上预测值与真实值的差距;
- L 越大表示这组参数越差, L 越小表示越好。

2. Define Loss from Training Data

- Loss is a function of parameters $L(b, w)$
- Loss: how good a set of values is.



$$\text{Loss: } L = \frac{1}{N} \sum_n e_n$$

$$e = |y - \hat{y}| \quad L \text{ is mean absolute error (MAE)}$$

$$e = (y - \hat{y})^2 \quad L \text{ is mean square error (MSE)}$$

图 5: 损失的一般式 $L = \frac{1}{N} \sum_n e_n$, 以及单笔误差 e 的两种常见定义: 绝对值 (MAE) 与平方 (MSE)

5

单笔误差 e 的计算方式不止一种:

- 取绝对值 $e = |y - \hat{y}|$, 对应的损失称为**平均绝对误差** (MAE, Mean Absolute Error);
- 取平方 $e = (y - \hat{y})^2$, 对应**均方误差** (MSE, Mean Square Error)。

MAE、MSE 与交叉熵怎么选

MAE 与 MSE 之间存在微妙差别, 选用哪一种取决于任务需求和你对问题的理解, 并非随意。特别地, 若 y 与 $\hat{y}$ 都是几率分布, 则常改用**交叉熵** (cross-entropy) 来衡量差距。

如果把每一组参数 (w, b) 都试一遍、各自算出损失 L , 就能把结果画成一张**误差曲面** (error surface) 等高线图。

⁵ 视频画面时间区间: 00:21:45-00:22:00。

2. Define Loss from Training Data

- Loss is a function of parameters $L(b, w)$
- Loss: how good a set of values is.

Model $y = b + wx_1$

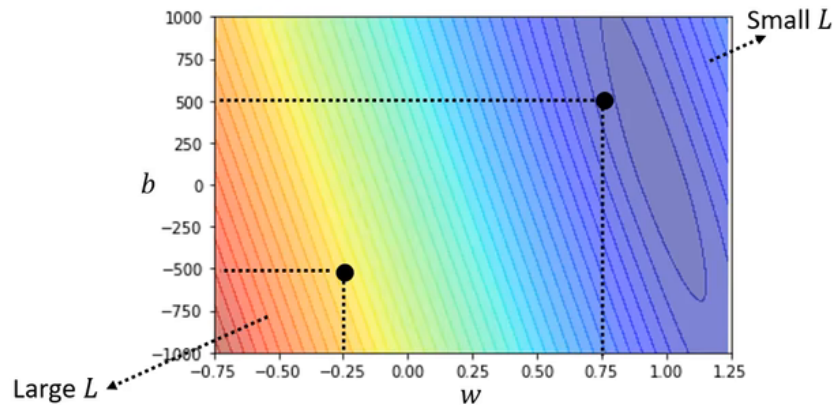


图 6: 误差曲面 (error surface): 横轴 w 、纵轴 b , 颜色表示损失大小。越偏红色 L 越大 (参数越差), 越偏蓝色 L 越小 (参数越好) ⁶

在这张真实数据画出的等高线图上, 损失最小 (最蓝) 的位置大约在 $w \approx 1$ 、 b 取一个较小的值附近。这其实符合直觉: 用前一天的观看数预测隔天, 二者往往差不多, 因此当 $w \approx 1$ 、 b 仅作小幅修正时, 预测最准。

误差曲面 error surface

把每一组参数对应的损失可视化等高线, 便得到 error surface。它直观地告诉我们“往哪个方向走、参数怎么调”才能让损失变小——而这正是第三步“最佳化”要解决的问题。

3.3 第三步: 最佳化 (Optimization)

第三步要解一个**最佳化** (optimization) 问题: 找出让损失最小的那组参数, 记作 w^*, b^* :

$$w^*, b^* = \arg \min_{w, b} L.$$

本课程使用的最佳化方法只有一种——**梯度下降** (Gradient Descent)。

为便于理解, 先假设只有一个参数 w , 此时 error surface 退化成一条一维曲线。梯度下降的做法是:

1. 随机选取一个初始值 w^0 ;
2. 计算损失对 w 的微分 (即该点切线的斜率) $\left. \frac{\partial L}{\partial w} \right|_{w=w^0}$;
3. 沿着“让损失下降”的方向, 把 w 挪动一步。

⁶视频画面时间区间: 00:24:30-00:24:45。

3. Optimization

$$w^* = \arg \min_w L$$

Gradient Descent

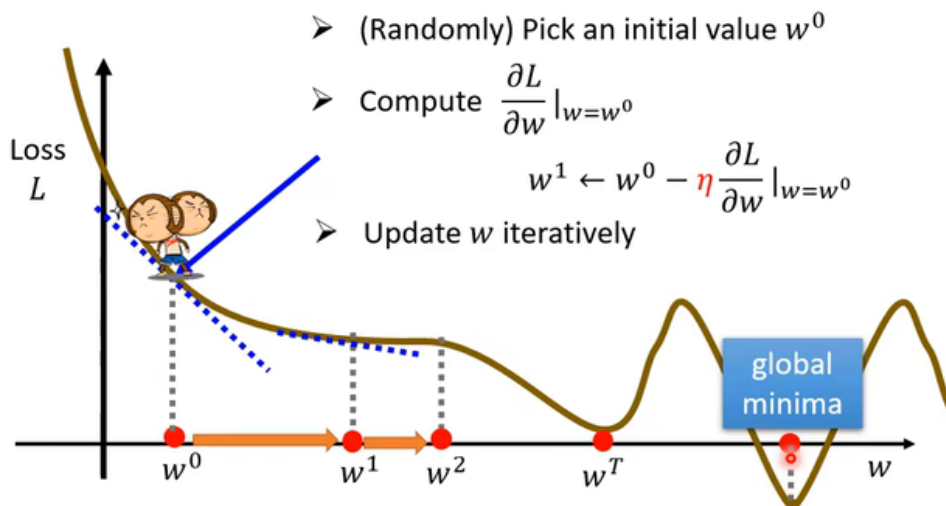


图 7: 梯度下降: 从随机初始点 w^0 出发, 按 $w^1 \leftarrow w^0 - \eta \frac{\partial L}{\partial w}$ 反复更新; 右侧标出了全局最小 (global minima) 与可能困住它的局部最小 (local minima) ⁷

斜率为负 (左高右低) 就增大 w , 斜率为正 (左低右高) 就减小 w 。形象地说, 就像一个人站在山坡上左右环视, 朝更低的方向跨出一步。这一步跨多大, 取决于两件事: 当前斜率的大小, 以及一个你自己设定的**学习率** (learning rate) η 。更新公式为:

$$w^1 \leftarrow w^0 - \eta \frac{\partial L}{\partial w} \bigg|_{w=w^0}$$

- η : 学习率, 由人自行设定, 控制每一步的幅度;
- $\partial L / \partial w$: 当前位置的斜率 (梯度);
- 负号: 保证朝损失**下降**的方向移动。

超参数 hyperparameter

像学习率 η 、以及“最多更新几次”这类需要**人为设定**、而非机器从资料中学出来的量, 统称为**超参数** (hyperparameter)。它与“参数” (w 、 b , 由机器学出) 是两个不同层级的概念。

反复迭代 $w^0 \rightarrow w^1 \rightarrow w^2 \rightarrow \dots$, 通常在两种情况下停止: (1) 达到预设的更新次数上限 (“失去耐心”, 次数本身也是超参数); (2) 微分恰好为 0, 参数不再移动。

⁷ 视频画面时间区间: 00:35:00–00:35:15。

Local minima 是个“假议题”

梯度下降一旦走到斜率为 0 处就会停下，因此可能停在并非全局最低的**局部最小** (local minima)，而非**全局最小** (global minima)。教科书常把这说成梯度下降的致命缺陷。但李宏毅老师特别提醒：在真实的深度学习里，local minima 其实是个“假议题”——梯度下降真正棘手的难题另有其物，留待后续课程揭晓（伏笔见第四节：局部最小与鞍点）。

推广到两个参数 w 、 b 时，做法完全一样：分别计算 $\partial L/\partial w$ 与 $\partial L/\partial b$ ，再同步更新二者。这些微分在 PyTorch 等深度学习框架里由一行程式自动算出，无需手算。

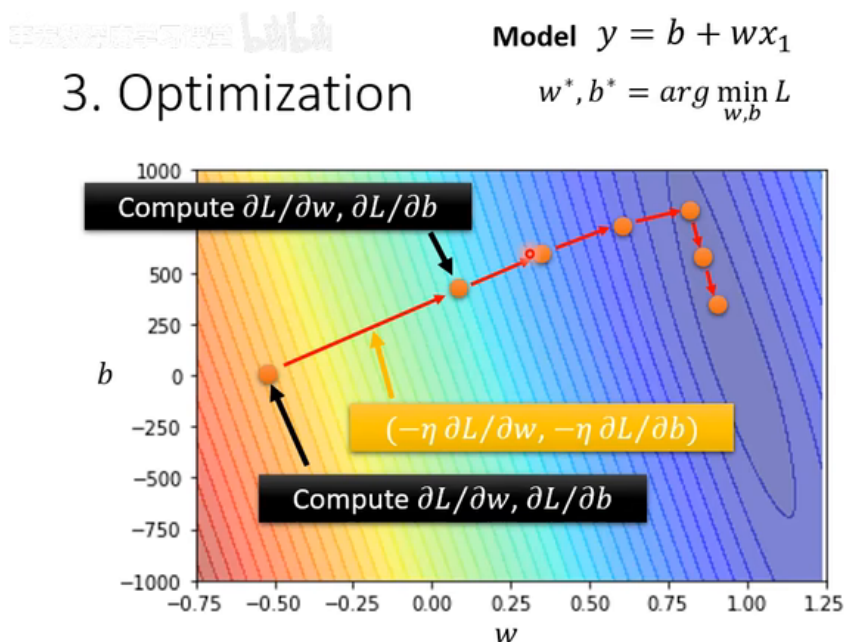


图 8: 双参数梯度下降：在 (w, b) 的误差曲面上，每一步沿 $(-\eta \frac{\partial L}{\partial w}, -\eta \frac{\partial L}{\partial b})$ 方向移动，逐步逼近损失更小的区域⁸

在真实数据上跑梯度下降，得到最佳参数 $w^* = 0.97$ 、 $b^* \approx 0.1k$ ，训练资料（2017–2020）上的损失 $L = 0.48k$ 。这与最初的猜测 $w \approx 1$ 相当接近——用前一天预测隔天，本就该有 $w \approx 1$ 。

3.4 本章小结

机器学习“找函数”的三个步骤至此完整：

⁸视频画面时间区间：00:39:30–00:39:45。

机器学习的三个步骤（

1. **Model**: 写出一个带未知参数的函数 $y = b + wx_1$;
2. **Loss**: 定义损失 $L(b, w)$, 用训练资料衡量参数好坏;
3. **Optimization**: 用梯度下降找出 w^*, b^* 使损失最小。

三步合起来称为**训练** (training)。

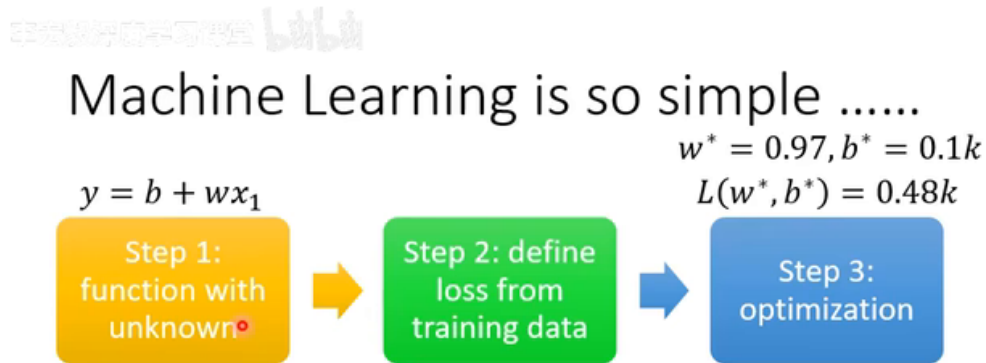


图 9: “Machine Learning is so simple” ——三步骤总览，以及本例求得的最佳参数 $w^* = 0.97$ 、 $b^* = 0.1k$ 、训练损失 $L = 0.48k$ ⁹

4 从训练到预测：泛化的考验

然而 $L = 0.48k$ 是在**已经知道答案**的训练资料上算出来的——用李宏毅老师的话说，这只是“自嗨”。我们真正关心的，是模型对**未来未知数据**的预测能力。于是用 2020 年底找到的函数，逐日预测 2021 年（1 月 1 日至 2 月 14 日）的观看数：在这些**没看过**的数据上，误差 $L' = 0.58k$ ，明显大于训练误差 $0.48k$ 。

⁹视频画面时间区间：00:40:30–00:40:45。

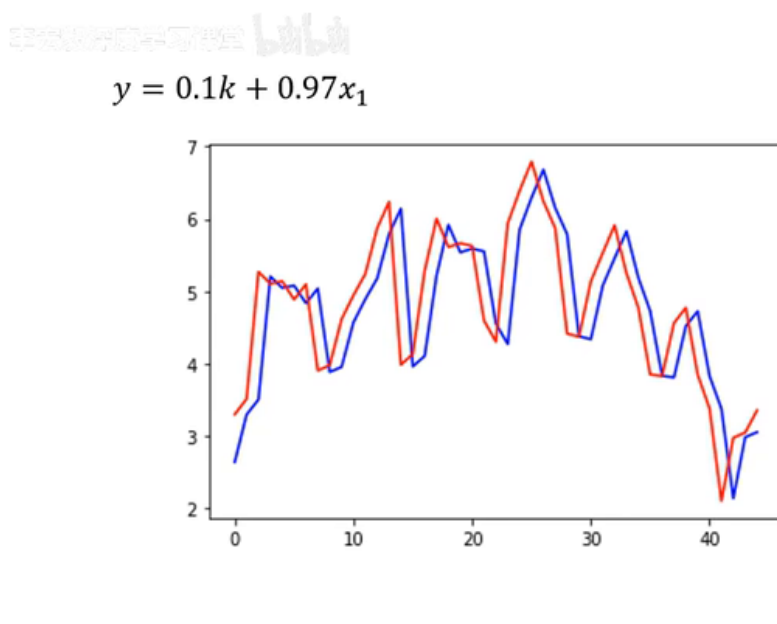


图 10: 2021 年预测结果：横轴为时间，红线是真实观看人次、蓝线是模型预测。蓝线几乎就是红线“向右平移一天”¹⁰

训练误差 $\neq$ 测试误差

在见过的资料上表现好，不代表在没见过的资料上也好。我们真正追求的是后者，即**泛化能力**。这是机器学习中必须时刻警惕的区别。

仔细观察预测曲线还能发现：蓝线几乎是红线右移一天——因为这个模型本质上就是“拿前一天预测隔天”（ $w \approx 1$ ）。而真实数据有一个明显的 **7 天周期**：每周五、周六观看数特别低（周末大家都出去玩，没人学机器学习）。

5 让模型更有弹性：线性模型

既然数据有 7 天周期，只看前一天的模型显然太弱。于是把前 7 天都纳入考虑——这次对模型的改良，又一次来自领域知识（domain knowledge）：

$$y = b + \sum_{j=1}^7 w_j x_j$$

其中 x_j 是 j 天前的观看数， w_j 是它对应的权重。

¹⁰ 视频画面时间区间：00:43:15–00:43:30。

$$y = b + wx_1 \quad \begin{array}{cc} 2017 - 2020 & 2021 \\ L = 0.48k & L' = 0.58k \end{array}$$

$$y = b + \sum_{j=1}^7 w_j x_j$$

图 11: 从单特征 $y = b + wx_1$ 扩展到多特征 $y = b + \sum_{j=1}^7 w_j x_j$: 把前若干天的观看数都作为特征纳入考虑¹¹

继续增加纳入考虑的天数，训练与测试损失的变化如下表：

纳入的天数	训练损失 L (2017–2020)	测试损失 L' (2021)
1 天	0.48k	0.58k
7 天	0.38k	0.49k
28 天	0.33k	0.46k
56 天	0.32k	0.46k

表 1: 考虑不同天数的线性模型，在训练与测试资料上的损失对比。

可以看到：纳入越多天，训练损失持续下降；但**测试损失**在 28 天之后就停在约 0.46k 不再改善——靠“多看几天”来提升，已经到了极限。

线性模型 Linear Model

上面这些模型都是“特征乘以权重、再加偏置”的形式：

$$y = b + \sum_j w_j x_j.$$

它们有一个共同的名字——**线性模型** (linear model)。

¹¹ 视频画面时间区间：00:47:00–00:47:15。

5.1 本章小结

借助 domain knowledge 引入更多特征（前 N 天），线性模型把测试误差从 0.58k 降到约 0.46k，但很快触及瓶颈。这暗示：线性模型的表达能力存在天然上限。

6 总结与延伸

讲者的收束：本节用一个贯穿始终的例子（预测频道观看数），完整走过了机器学习的三个步骤，并亲眼看到线性模型的能力极限。李宏毅老师在结尾点明下一步——“看看怎么把 linear model 做得更好”。这正是通往更复杂模型、乃至深度学习的起点。

结构化提炼：

- **一条主线：**机器学习 = 找一个函数；而“找函数”被拆解为三步骤——Model $\rightarrow$ Loss $\rightarrow$ Optimization。
- **两个关键区分：**参数 (w, b ，机器学出) 与超参数 (η 、迭代次数等，人为设定)；训练误差与测试误差（真正重要的是后者，即泛化）。
- **一个伏笔：**梯度下降的 local minima 被老师称为“假议题”，真正的难题留待后续揭晓。
- **一处局限：**线性模型即便不断增加特征，也存在天花板，需要更有弹性的模型。

下一步：第一节（下）将引入 Sigmoid / ReLU，把线性模型扩展成能逼近任意曲线的函数，进而引出“类神经网络”与深度学习的基本概念。